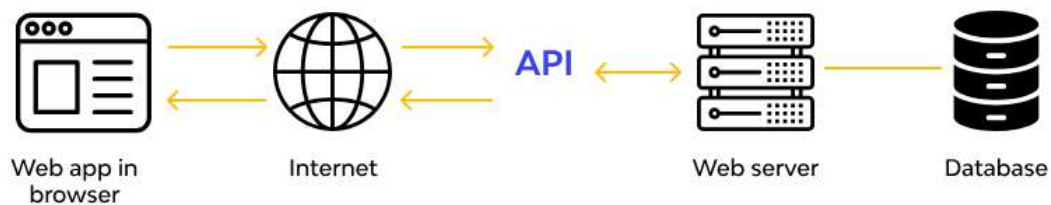


1. What is an API?

- API (Application Programming Interface) is a way that allows two different software applications to communicate and exchange data with each other.
- API is used to communicate between the frontend and backend and exchange data.

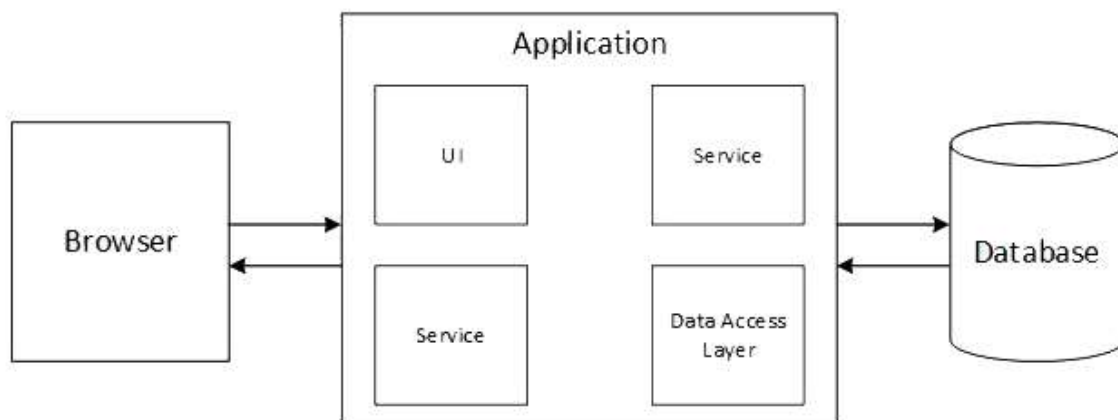


2. Why Do We Need an API?

- We need an API to allow the frontend and backend to communicate and exchange data.

3. What Did We Use Before APIs?

- Before modern APIs, applications commonly used direct communication between the client and server, where the client was tightly connected to the server-side application.
- The client would directly request information from the server, and the server would process the request and return a complete page or response.



Example

- 1) **PHP** – PHP + MySQL
- 2) **Java** – JSP/Servlets
- 3) **Python** – Django

4. Limitations of Tightly Coupled Architecture

- **Difficult to modify** – Changing one part may require changes in other parts.
- **Difficult to scale** – You cannot easily scale frontend and backend independently.
- **Difficult to maintain** – Large applications become harder to manage.
- **Less flexibility** – The same backend cannot easily be used by different clients.
- **Difficult to reuse** – Business logic is often tied to a specific application.

Ex

- e-commerce Website
 - |— User Management
 - |— Product Management
 - |— Order Management
 - |— Payment

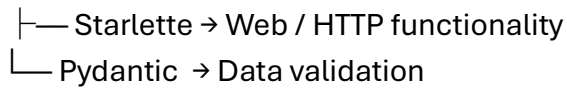
Everything is inside **one application**.

- If the Payment module has a problem, the entire application may be affected. Even if you only want to update the Payment module, you may need to deploy the whole application again.
- If another website wants to display those products for advertising we can not able to display.

5. What is FastAPI?

- FastAPI is a modern Python web framework used to build APIs and backend applications quickly and efficiently.
- FastAPI is built using Starlette for web functionality and Pydantic for data validation and serialization.

FastAPI



1) Starlette

- Starlette is a lightweight Python framework/toolkit used for building web applications and handling HTTP requests and responses.
- It handles things like routes, requests, responses, middleware, and HTTP communication in FastAPI.

2) Pydantic

- Pydantic is a Python library used for data validation and data parsing.
- It checks whether the data received by the API is in the **correct format and type**.

6. Why Do We Use FastAPI?

- FastAPI is used to build APIs and backend applications quickly, efficiently, and easily using Python.

Main Reasons

- Fast performance
- Easy to learn and use
- Automatic data validation using Pydantic
- Easy to build REST APIs
- Supports asynchronous programming

7. What Did We Use Before FastAPI?

- Before FastAPI, we commonly used frameworks like **Flask** and **Django** to build web applications and APIs.

Limitations

- **Flask:** Simple and flexible, but many features need to be added/configured separately.
- **Django:** Powerful and feature-rich, but can be heavy for small API-focused applications.
- **Data validation:** Not as naturally integrated as FastAPI + Pydantic.
- **Async support:** Traditional Flask/Django approaches were not designed primarily around modern asynchronous APIs.

8. Why is FastAPI Fast?

- FastAPI is considered **fast in two ways**:

1) Fast in Execution

- FastAPI supports asynchronous programming.
- It can efficiently handle multiple requests without blocking while waiting for operations such as database or network requests.

2) Fast in Development

- FastAPI helps developers build APIs faster because it provides:
 - **Pydantic** → Automatic data validation
 - **Automatic API documentation** → Swagger UI and ReDoc
 - **Type hints** → Better code clarity and validation
 - **Modern Python features** → async / await
 - **Less boilerplate code** → Faster API development

9. FastAPI Setup

- Create a Project Folder
- Create Virtual Environment
`python -m venv venv`
- Activate venv
`venv\Scripts\activate`
- Install FastAPI and Uvicorn
`pip install fastapi uvicorn`
- Create main.py
- Run the Server
`uvicorn main:app --reload`

10. What is Postman?

- Postman is an API testing tool used to send requests to APIs and check their responses.
- It allows developers to test GET, POST, PUT, and DELETE requests without needing a frontend application.
- HTTP Status Codes

Code	Meaning	Use
200	OK	Request successful
201	Created	New data successfully created
400	Bad Request	Invalid request/data
401	Unauthorized	Authentication required/invalid

Code	Meaning	Use
403	Forbidden	Access denied
404	Not Found	Requested resource not found
500	Internal Server Error	Server-side error